ASCEND — Defensa de Programación Full Stack

Materia: Programación Full Stack · Prof. Pablo Morales · 3°MM · 2026 **Proyecto:** ASCEND — Sistema de Gestión Deportiva Modular (SGDM) **Uso:** banco de respuestas para la defensa oral. Cada pregunta trae tres capas: qué decir, con qué evidencia y cuál es el límite si repreguntan.

---

0. Cómo usar este documento

Tres reglas, en este orden:

1. **Respuesta corta primero.** 30 a 60 segundos. Si el profesor quiere más, pregunta; no entregues todo de entrada.
2. **Cada afirmación tiene un archivo o una consulta detrás.** Los números de la ficha técnica se verifican en vivo con los comandos del Anexo C.
3. **El límite se declara antes de que lo encuentren.** "Eso no está implementado" vale más que una respuesta inventada. Si no lo medimos, se dice "no lo medimos".

Y una advertencia: este documento distingue lo que **está en el código** de lo que **está solo en la documentación de planificación**. Esa diferencia es la que suele separar una defensa sólida de una que se cae en la segunda pregunta.

Ficha técnica verificada

Estos son los números que se pueden decir sin riesgo. Todos salen de una consulta a la base en ejecución, de un conteo sobre el código o del informe de verificación final.

Dato	Valor	Cómo se comprueba
Tablas	**41**	`information_schema.tables`
Migraciones	**7** (`001` a `007`)	`base_de_datos/migraciones/`
Claves foráneas	**57**	`information_schema.referential_constraints`
Políticas de FK	36 `CASCADE`, 14 `SET NULL`, 7 `RESTRICT`	misma consulta, agrupada por `DELETE_RULE`
Restricciones `CHECK`	**11**	`table_constraints`
Índices secundarios	**70** (79 entradas de columna)	`information_schema.statistics`
Triggers	**2** (validan disciplina/modalidad/formato en `torneos`)	`information_schema.triggers`
Motor	**InnoDB**	`information_schema.tables`

Dato	Valor	Cómo se comprueba
PHP / MySQL	**8.2.29** (imagen `php:8.2.29-apache`) / **8.0.46**	`docker/Dockerfile`, `docker/docker-compose.yml`
Rutas verificadas por HTTP	**50** con 200 (16 públicas, 18 admin, 8 organizador, 8 jugador)	`informes_auditoria/VERIFICACION_FINAL_DRY_RUN.md`
Consultas preparadas	**238** llamadas a `prepare()` en modelos	conteo sobre `codigo_fuente/modelos`
Escapes de salida	**371** `htmlspecialchars` en vistas	conteo sobre `codigo_fuente/vistas`
Guardas de rol	**37** llamadas a `Sesion::validar*` / `requerirLogin`	conteo en controladores
Guardas CSRF	**42** llamadas a `requerirTokensCSRF` / `validarTokenCSRF`	conteo en controladores
Formularios con token	**65** campos `csrf_token`	conteo en vistas
Bloqueos pesimistas	**10** `SELECT ... FOR UPDATE`	conteo en modelos
Pruebas automatizadas	**7 tests, 25 assertions, OK** (PHPUnit 9.6.36 / PHP 8.0.30)	informe de dry run, sección 5
Datos de demostración	52 usuarios, 20 torneos, 5 juegos, 23 equipos, 81 participantes, 87 encuentros	consulta en vivo (el seed base crea 51 usuarios y 18 torneos; hay dos torneos de prueba agregados después)

Si algún número no coincide en el momento de la defensa, gana la consulta en vivo, no este archivo.

Nota de consistencia documental: el `README.md` todavía lista las migraciones `001`–`006`; la `007_restricciones_disciplinas.sql` se agregó el 24/10 y sus dos triggers están aplicados en la base actual. Si el profesor abre el README y ve seis, la respuesta es esta: "son siete archivos; el README quedó desactualizado y la 007 es la que valida disciplina, modalidad y formato".

I. Arquitectura y conceptos generales Full Stack

I.1. ¿Cómo explicas la arquitectura general de tu sistema a alguien no técnico?

Decí esto. "ASCEND es como la mesa de control de un torneo. Hay una cartelera pública que cualquiera puede mirar, una ventanilla donde el jugador se inscribe, una cabina donde el organizador carga resultados y un libro de actas donde queda todo registrado. La clave es que los cuatro miran la misma cartelera y escriben en el mismo libro: no existen cuatro versiones del torneo. Por debajo, el sistema tiene una sola puerta de entrada, un archivo central que es la base de datos y un conjunto de oficinas internas que hacen cada trámite."

Si piden el equivalente técnico. "Es un MVC clásico: todo request entra por `publico/index.php`, que decide qué controlador atiende; el controlador valida sesión y permisos, llama al modelo, y el modelo es el único que habla con MySQL mediante PDO. La vista devuelve HTML distinto según el rol. Arriba hay Docker para que el entorno se reconstruya igual en cualquier máquina."

Evidencia. `codigo_fuente/publico/index.php:38-113` (whitelist y despacho), `codigo_fuente/modelos/Conexion.php:16-38` (PDO único), `docker/docker-compose.yml`.

Límite. No es una arquitectura de microservicios ni tiene capa de servicios: es un monolito modular. Para el tamaño del proyecto es una ventaja, no una disculpa.

I.2. ¿Cuál es la diferencia entre CSR y SSR en tu proyecto?

Decí esto. "Nuestro renderizado es del lado del servidor. Cada request entra por `index.php`, el controlador consulta MySQL y la vista devuelve el HTML ya armado con los datos. El navegador recibe una página completa, no un esqueleto vacío que se llena después. El JavaScript no construye la pantalla principal: mejora algunas piezas —el navbar y el footer públicos se traen por `fetch`— y resuelve una búsqueda AJAX de jugadores. No usamos un SPA ni un framework de componentes."

Por qué elegimos SSR. El dominio es formularios, listados y tablas de posiciones. SSR da primera pintura inmediata, funciona sin build y permite que el HTML sea indexable. CSR habría sumado complejidad de estado y un pipeline de compilación que no aportaba al problema.

Evidencia. `codigo_fuente/publico/js/includesPublicos.js:31` y `:77` (navbar/footer por `fetch`), `codigo_fuente/publico/js/buscar-jugadores-vivo.js:62` (búsqueda AJAX), vistas PHP que imprimen datos del modelo.

Límite. Hay una isla de CSR que conviene reconocer: si el usuario desactiva JavaScript, la página pública pierde el navbar y el footer inyectados. Es una deuda de diseño, no un comportamiento accidental.

I.3. ¿Cómo se comunican el frontend y el backend?

Decí esto. "Hablan HTTP. La navegación son formularios GET y POST contra un único front controller con dos parámetros: `c` para el controlador y `a` para la acción. Los POST llevan token CSRF y se validan en el servidor. Para la búsqueda de jugadores, el mismo servidor responde JSON y el JS lo pinta. No hay una API REST separada: el front controller sirve HTML para las páginas y JSON para el endpoint AJAX."

Ejemplo concreto. Un formulario de inscripción hace `POST index.php?c=torneo&a=inscribirse` con `csrf_token`; el controlador verifica sesión, token, cupo y duplicados; el modelo inserta en `participantes_torneo`; el servidor responde con una redirección y un mensaje flash. La búsqueda hace `GET index.php?c=equipo&a=buscarJugadoresAjax&id=...&buscar=...` y recibe un array JSON.

Evidencia. `codigo_fuente/controladores/EquipoControlador.php:384-399` (respuesta JSON), `index.php:38-67` (rutas), 65 campos `csrf_token` en vistas.

Límite. No hay contrato formal de API ni OpenAPI. Si un tercero tuviera que consumir el sistema, habría que separar endpoints y versionarlos.

I.4. ¿Cómo maneja tu sistema la escalabilidad si el tráfico aumenta repentinamente?

Decí esto. "Hoy es un monolito PHP con MySQL. La aplicación escala horizontalmente con relativa facilidad porque no guarda estado propio en memoria: se pueden levantar varias instancias del contenedor web detrás de un proxy. Los dos cuellos de botella reales son la sesión, que vive en archivos dentro de cada contenedor, y MySQL, que tiene un solo maestro. El diseño ya contempla réplica de lectura y el toolkit de Sistemas Operativos automatiza una replicación maestro-esclavo. Lo que no hay es caché, balanceador ni prueba de carga."

Qué se rompería primero. Con varias instancias web, un usuario podría autenticarse en una y perder la sesión en la siguiente. La solución es mover sesiones a un almacén compartido (Redis o tabla de sesiones) o usar sesiones pegajosas. Después, el cuello pasa a MySQL: ahí entran réplica de lectura para catálogo y rankings, índices y caché de consultas pesadas.

Evidencia. `docker/docker-compose.yml` (servicios y healthchecks), `codigo_fuente/ayudantes/Sesion.php:24-35` (sesión nativa en archivos), `administracion_sistemas/` (replicación), 70 índices secundarios en la base.

Límite. No hicimos prueba de carga. No puedo afirmar "soporta N usuarios"; puedo explicar qué componente falla primero y cómo se mitiga.

I.5. ¿Dónde reside la lógica de negocio principal y por qué la ubicaron allí?

Decí esto. "La lógica vive en los modelos, y ahí está la razón: las reglas que no pueden quedar a medias necesitan transacción y una sola implementación. Pasar un torneo de borrador a inscripciones, generar un fixture, recalcular posiciones o aprobar una solicitud de equipo son operaciones de varios pasos. Si estuvieran en la vista, cualquiera podría saltearlas; si estuvieran duplicadas en dos controladores, se desincronizarían. El controlador solo autentica, valida la entrada y redirige; la vista solo muestra."

Ejemplo concreto. Al guardar un resultado, el modelo bloquea el encuentro, valida que el torneo esté en curso, actualiza el marcador y recalcula la tabla de posiciones dentro de la misma transacción. Si algo falla, hace `rollBack` y no queda una posición a medio actualizar.

Evidencia. `codigo_fuente/modelos/Torneo.php:1082-1088` (transacción y `FOR UPDATE`), `codigo_fuente/modelos/Equipo.php:295` y `:503` (bloqueo al responder solicitudes), controladores con guardas y redirecciones.

Límite. Hay validación de formato en los controladores y reglas de estado en los modelos; no existe una capa de servicios intermedia. En un equipo más grande convendría separarla, pero para este alcance evita una indirección que no aportaba.

II. Backend y lógica del servidor

II.1. ¿Qué lenguaje y framework usaste en el backend y cuáles son sus ventajas?

Decí esto. "PHP 8.2 sobre Apache, sin framework. La decisión fue de alcance y de la materia: queríamos controlar y poder explicar el flujo completo. La ventaja es que no hay magia: el routing, la sesión, el CSRF y el acceso a datos están escritos y son auditables. El costo es más código propio y menos herramientas listas: no

hay ORM, no hay migraciones automáticas, no hay generador de CRUD. Composer se usa solo para PHPUnit en desarrollo."

Ventajas concretas de PHP para este caso. Está pensado para request-response, se integra directo con Apache y MySQL, el hosting es trivial y la curva del equipo ya lo cubría. La imagen de Docker pesa poco y el arranque es reproducible.

Evidencia. `docker/Dockerfile` (`php:8.2.29-apache`), `composer.json` (solo `phpunit` en `require-dev`), `index.php` como front controller manual.

Límite. No afirmamos que sea mejor que Laravel o Symfony. Un framework habría dado ORM, middleware y validación listos; habría costado menos código y más caja negra. Fue un trade-off explícito.

II.2. ¿Cómo estructuraste las rutas y controladores en el código del servidor?

Decí esto. "Hay un único punto de entrada. El request trae `c` y `a`; el front controller busca esa combinación en una lista blanca y, si no está, devuelve 404 antes de cargar cualquier clase. Si está, arma el nombre del controlador con una convención —`ucfirst($c) . 'Controlador'`—, verifica que el archivo exista, lo instancia y llama al método. Cada acción vuelve a validar sesión y rol. No hay rutas mágicas ni `include` dinámico arbitrario: la lista blanca es la frontera."

Por qué así. Con un solo entrypoint se aplican las cabeceras de seguridad y el manejador de errores una sola vez, y el docroot puede quedar limitado a `publico/`, dejando configuración, modelos y vistas fuera del alcance web.

Evidencia. `codigo_fuente/publico/index.php:38-67` (whitelist), `:84-113` (despacho y 404), `docker/apache-vhost.conf` (`DocumentRoot /var/www/html/publico`).

Límite. La lista blanca se mantiene a mano: agregar una acción exige editar dos archivos. Es explícito, pero es trabajo repetitivo y un olvido produce un 404, no un agujero.

II.3. ¿Cómo gestionas el manejo de errores en el backend para evitar que el servidor se caiga?

Decí esto. "Hay un manejador global registrado en el front controller: `set_exception_handler` y `set_error_handler`. Cuando hay una excepción, se registra en la tabla `errores_sistema` con archivo, línea, stack trace, usuario y contexto; el contexto se censura antes de guardarse —contraseñas y tokens se reemplazan por asteriscos—. Al usuario se le devuelve una página 500 genérica, no el detalle técnico. Si el mismo error se repite, no se duplica: se incrementa un contador por hash. Y si la base está caída y no se puede registrar, cae al `error_log` nativo de PHP."

Por qué importa. Una excepción no manejada deja una pantalla blanca o filtra rutas internas. Acá el buffer se limpia, se responde 500 y queda evidencia para el panel de monitoreo.

Evidencia. `codigo_fuente/publico/index.php:20-25` (registro), `codigo_fuente/core/ManejadorErrores.php:11-124` (registro, censura, 500), `ob_start()` en la primera línea del front controller.

Límite. Los *warnings* de PHP se registran, pero no se convierten en excepciones; no hay alertas automáticas ni reintentos. Tampoco hay un `php.ini` propio en la imagen: el endurecimiento de `display_errors` y

`error_reporting` para producción queda pendiente.

II.4. ¿Cómo validas los datos entrantes antes de procesarlos o guardarlos?

Decí esto. "En cuatro capas. Primero, HTML5 en el navegador: `required`, `type=email`, longitudes. Eso es experiencia de usuario, nunca una garantía. Segundo, el controlador: verifica método HTTP, token CSRF, campos obligatorios, formato de email con `filter_var`, listas blancas con `in_array` y casts explícitos a entero o flotante. Tercero, el modelo: vuelve a comprobar reglas de negocio —por ejemplo, que el torneo esté en el estado correcto o que el cupo no esté lleno—. Cuarto, la base: 57 claves foráneas, 11 `CHECK`, claves únicas y `NOT NULL` como última red."

Ejemplo. Un `id` que llega por GET se castea a entero; un estado que llega por POST se compara contra una lista; un `ORDER BY` no se concatena: se elige con un `switch` entre valores permitidos.

Evidencia. 76 atributos `required`, 21 usos de `in_array` y 16 casts directos sobre `$_GET`/`$_POST` relevados en controladores, 9 `filter_var`, `codigo_fuente/modelos/Torneo.php` (filtros con parámetros enlazados y `switch` de orden), `base_de_datos/migraciones/001_ascend.sql`.

Límite. No hay un validador central. `codigo_fuente/ayudantes/Validador.php` está vacío: cada controlador repite sus comprobaciones. Funciona, pero es deuda técnica y admite inconsistencias entre formularios.

II.5. ¿Cómo implementaste la paginación y filtrado en endpoints que devuelven listas grandes?

Decí esto. "Paginación no hay. No existe `OFFSET` en el código; lo verifiqué. Los listados están acotados por el dominio —5 juegos, 20 torneos, 23 equipos— y las consultas que podrían crecer llevan `LIMIT`: la búsqueda de jugadores devuelve 20, el ranking global 10. El filtrado sí está resuelto en servidor: el catálogo filtra por juego, estado y orden, con parámetros enlazados y una lista blanca para el orden."

Por qué lo digo sin rodeos. Si el profesor pregunta por paginación, la respuesta honesta es que no está implementada. El sistema asume catálogos chicos y una demo académica. Inventar un "sí, paginamos" se cae en la primera repregunta.

Cómo se agregaría. Para listados administrativos: `COUNT(*)` para el total, `LIMIT :limite OFFSET :offset` con ambos valores casteados, y parámetros de página validados. Para listas muy grandes o scroll infinito: paginación por keyset (`WHERE id < :ultimo ORDER BY id DESC LIMIT N`), que evita el costo de los offsets altos.

Evidencia. Búsqueda con `LIMIT 20` en `codigo_fuente/modelos/Equipo.php:352`; ranking con `LIMIT 10` en `ResultadoGlobal.php:44`; `switch` de orden en `Torneo.php` (`obtenerDirectorioAdmin`); cero ocurrencias de `OFFSET` en todo el proyecto.

Límite. Es una deuda declarada, no un olvido. Con 41 tablas y este volumen no afecta la demo; con miles de filas habría que resolverlo antes de producción.

II.6. ¿Qué patrón de diseño aplicaste en el código del servidor?

Decí esto. "El patrón estructural es MVC. Además hay cuatro patrones concretos: *Front Controller*, porque todo entra por `index.php`; *Singleton*, porque `Conexion` garantiza una sola instancia de PDO por request; *Table Data Gateway*, porque cada modelo encapsula las consultas de su tabla y devuelve arrays; y *Template View*, porque las vistas PHP reciben datos y renderizan. Las guardas de sesión funcionan como un patrón *Guard*: cada acción verifica rol antes de ejecutar."

Por qué el Singleton. Evita abrir una conexión nueva en cada modelo. Su crítica conocida es el estado global y la dificultad de testear; en este proyecto se mitiga porque la conexión se inyecta implícitamente y los tests usan una base desechable.

Evidencia. `codigo_fuente/modelos/Conexion.php:41-48` (instancia única), `index.php` (front controller), `codigo_fuente/modelos/Torneo.php` (consultas encapsuladas), `codigo_fuente/ayudantes/Sesion.php` (guardas).

Límite. No usamos inyección de dependencias ni ORM. Los modelos mezclan acceso a datos y reglas de negocio; en un proyecto mayor convendría separar repositorio y servicio.

II.7. Si una petición tarda demasiado, ¿cómo diagnosticas qué parte del backend la genera?

Decí esto. "Primero reproduzco y mido: `curl -w` para separar tiempo total, de conexión y de primera respuesta. Después miro los logs de Apache y PHP del contenedor. Si el error quedó registrado, el panel de monitoreo muestra mensaje, archivo, línea y cantidad de ocurrencias. Si la demora es de base, entro a MySQL y reviso `SHOW FULL PROCESSLIST` para ver consultas activas y bloqueos, y corro `EXPLAIN` sobre la consulta sospechosa para ver si usa índice o hace un escaneo completo. Con eso se distingue si el costo está en PHP, en la consulta o en la red."

Qué buscaría. Consultas sin índice en columnas de filtro, `N+1` —muchas consultas chicas en un bucle—, tablas sin `WHERE`, locks esperando por una transacción larga, o una subida de archivo pesada.

Evidencia. `errores_sistema` y el panel `errorTracking`; 70 índices secundarios; transacciones con `FOR UPDATE` que pueden generar esperas; logs de Apache en `docker/apache-vhost.conf`.

Límite. No hay APM, profiler ni *slow query log* activado. El diagnóstico es manual. Para producción lo primero sería habilitar el slow log y agregar métricas por ruta.

III. Base de datos y persistencia

III.1. ¿Qué es la normalización y hasta qué forma normal llega tu diseño?

Decí esto. "Normalizar es organizar las tablas para que cada dato viva una sola vez y no haya redundancia que después se contradiga. El diseño está planteado en tercera forma normal: atributos atómicos, sin grupos repetitivos, sin dependencias parciales de claves compuestas y sin dependencias transitivas. Además usa herencia de tablas: `usuarios` es la tabla padre con identidad y credenciales, y `perfiles_jugadores` y `perfiles_organizadores` son hijas con la misma clave primaria, que es a la vez foránea. Así no hay columnas fantasma ni perfiles huérfanos."

Dónde se ve. Catálogos separados (`roles`, `permisos`, `juegos`, `modalidades`, `formatos_torneo`) y tablas puente para relaciones muchos a muchos (`juego_modalidades_permitidas`, `juego_formatos_permitidos`, `rol_permisos`, `equipo_miembros`).

La excepción controlada. `torneo_posiciones` es una tabla derivada: guarda la tabla de posiciones calculada para que el ranking se lea rápido, en lugar de recalcularlo en cada visita. Es redundancia intencional y regenerada por el sistema, no una fuente de verdad paralela.

Evidencia. `base_de_datos/migraciones/001_ascend.sql:1-32` (encabezado con las decisiones de 3FN y herencia), `:215-265` (usuarios y perfiles), tablas puente en el mismo archivo.

Límite. Hay columnas `JSON` en `auditoria_cambios` y `errores_sistema`. Ahí el JSON funciona como documento de contexto, no como reemplazo de un modelo relacional; no lo presentaría como "cuarta forma normal".

III.2. ¿Cómo gestionas las claves primarias y foráneas para mantener la integridad referencial?

Decí esto. "Cada tabla tiene una clave primaria sustituta: `id INT AUTO_INCREMENT`. Los datos naturales que deben ser únicos —el email, por ejemplo— llevan restricción `UNIQUE` aparte. Hay 57 claves foráneas declaradas, y las políticas no son todas iguales: 36 `ON DELETE CASCADE` para dependencias que no tienen sentido sin el padre, como torneo→encuentros; 14 `ON DELETE SET NULL` para registros históricos que deben sobrevivir, como una auditoría cuyo usuario se elimina; y 7 `RESTRICT` para catálogos críticos, como un rol con usuarios asignados. Las tablas puente usan clave primaria compuesta."

Por qué en la base y no solo en el código. Porque la integridad tiene que valer también para una carga manual, un script o un error de programación. La base es la última frontera.

Evidencia. `information_schema.referential_constraints` (57 filas; 36/14/7 por política), 9 claves primarias compuestas relevadas (`equipo_miembros`, `equipo_capitanes`, `rol_permisos`, `torneo_posiciones`, `torneo_suizo_parejas`, entre otras), `001_ascend.sql`.

Límite. Las cascadas de `torneos` además se ejecutan a mano en el modelo, en un orden explícito, dentro de una transacción (`Torneo.php:2010-2030`). Es correcto, pero duplica en código lo que la base ya podría hacer; se hizo así para controlar el orden y el borrado de tablas sin FK directa.

III.3. ¿Qué ocurre si dos usuarios modifican el mismo registro simultáneamente?

Decí esto. "En MySQL con InnoDB cada transacción bloquea las filas que toca. En las operaciones sensibles usamos bloqueo pesimista: `SELECT ... FOR UPDATE`. Por ejemplo, cuando dos capitanes intentan responder la misma solicitud, la transacción bloquea la fila, vuelve a verificar que siga pendiente y recién ahí decide. El segundo ve el estado ya cambiado y no duplica la operación. Lo mismo pasa al generar un fixture o cambiar el estado de un torneo: se bloquea el torneo para que dos organizadores no lo transicionen a la vez."

Qué no cubrimos. En ediciones administrativas simples —cambiar el nombre de un torneo— no hay control de concurrencia optimista con número de versión. Si dos administradores guardan a la vez, gana el último que escribe y el primero no se entera. Para el alcance actual es aceptable, pero es una diferencia entre "consistente" y "sin pérdidas silenciosas".

Evidencia. 10 `SELECT ... FOR UPDATE` (`Equipo.php:295`, `:503`, `:586`; `Torneo.php:1082-1088`, `:1319`, `:1380`, `:1996`), transacciones con `beginTransaction`/`commit`/`rollBack` en cinco modelos, claves únicas que impiden duplicados.

Límite. No hay bloqueo optimista ni aislamiento `SERIALIZABLE`. Tampoco pruebas de concurrencia automatizadas; la garantía se apoya en InnoDB y en el orden de las operaciones.

III.4. ¿Cómo implementaste el borrado: físico o lógico?

Decí esto. "Depende de la entidad, y la regla es simple: si el dato tiene historia o se referencia desde otro lado, se desactiva; si es una entidad transaccional que se quiere eliminar por completo, se borra físicamente con cascada controlada. Los usuarios se borran físicamente y sus perfiles, sesiones y tokens caen por cascada. Los torneos se borran físicamente en una transacción que elimina primero encuentros, posiciones, actividades y participantes. En cambio, equipos, juegos, formatos y la cuenta de usuario tienen banderas de activo: `equipos.activo`, `juegos.activo`, `usuarios.esta_activo`, y las consultas filtran por ellas."

Por qué. Un torneo cancelado no debería desaparecer del historial; un juego retirado no debería romper torneos viejos. El borrado lógico preserva la trazabilidad. El borrado físico se reserva para datos que no deben quedar.

Evidencia. `codigo_fuente/modelos/Usuario.php:259` (`DELETE FROM usuarios`), `Torneo.php:2010-2030` (cascada manual), `Equipo.php:604` (baja de capitán), banderas `activo`/`esta_activo` en `equipos`, `juegos`, `formatos_torneo`, `usuarios`, `newsletter_subscriptores`.

Límite. El borrado de torneo se dispara por GET y sin token CSRF (`torneoAdmin/eliminar`); la confirmación es un `confirm()` del navegador, que no es un control de servidor. Es una deuda de seguridad conocida y está en la lista de límites.

III.5. Si la base crece a millones de registros, ¿qué estrategia de optimización o particionamiento propondrías?

Decí esto. "Primero mediría antes de tocar: `EXPLAIN` de las consultas más frecuentes y *slow query log*. Después, en orden de costo-beneficio: índices compuestos que cubran los filtros reales —torneo, estado, fecha—; paginación por keyset en lugar de `OFFSET` alto; archivar torneos cerrados en tablas históricas; y recién entonces particionar. Para MySQL 8, un particionamiento por rango de fechas tiene sentido en `torneo_encuentros` o en logs, no en `usuarios`. La lectura se puede repartir con réplica; la escritura sigue en el maestro."

Qué evitaría. Particionar por una clave que no se usa en los `WHERE` obliga a escanear todas las particiones. Y agregar índices sin medir empeora las escrituras.

Evidencia. 70 índices secundarios; consultas de catálogo y ranking ya acotadas con `LIMIT`; replicación maestro-esclavo documentada en `administracion_sistemas/`; ausencia de particionamiento en el esquema.

Límite. Nada de esto está implementado ni probado a esa escala. Es un plan técnico, no una medición.

IV. Frontend y interfaz de usuario

IV.1. ¿Qué tecnología o framework de frontend usaste y por qué?

Decí esto. "HTML5, CSS3 con variables y JavaScript ES6, sin framework ni build. No hay React, Vue ni Tailwind; los estilos son propios y las plantillas son PHP que renderiza el servidor. Elegimos eso por tres razones: el

equipo ya dominaba la base web, el proyecto no necesitaba reactividad compleja, y un pipeline de compilación habría agregado una dependencia que no aportaba al problema. Los íconos vienen de Font Awesome y las tipografías de Google Fonts."

El costo. Todo el DOM se maneja a mano y no hay componentes con estado propio; la consistencia depende de convenciones y de las variables CSS.

Evidencia. 47 archivos CSS y 120 media queries en `codigo_fuente/publico/css`; `variables.css` con tokens de color, radios y espaciado; JS modular en `publico/js`; vistas PHP.

Límite. No hay guía de estilo formal ni sistema de componentes documentado. La coherencia visual se sostiene con tokens y revisión, no con una librería.

IV.2. ¿Cómo garantizas que la interfaz sea responsive?

Decí esto. "Con enfoque mobile-first: los estilos base son para pantalla chica y los `@media` agregan complejidad con `min-width`. Hay 120 media queries distribuidas en 47 archivos, con cortes en 600, 700, 900 y 1100 píxeles. Las grillas y los paneles usan flex y grid, y el layout administrativo tiene su propia hoja con cortes en 768. Las 34 páginas completas declaran `meta viewport`; las demás son fragmentos que se incluyen dentro de un layout que ya lo declara."

Por qué el detalle de los fragmentos. Si alguien cuenta 61 archivos de vista y solo 34 con viewport, la explicación es esa: 18 son secciones del panel admin que se inyectan en `dashboard.php`, y el resto son navbar, footer y menú lateral. No son páginas independientes.

Evidencia. `codigo_fuente/publico/css/variables.css`, hojas por área (`publico`, `jugador`, `organizador`, `admin`), `vistas/admin/dashboard.php:9` (viewport del layout), conteo de media queries.

Límite. Lo verificamos en navegador de escritorio y en algunos anchos móviles, no en una matriz de dispositivos reales. No hay pruebas visuales automatizadas.

IV.3. ¿Cómo manejas la carga asíncrona para que el usuario no vea pantallas en blanco?

Decí esto. "La mayor parte de la pantalla llega renderizada del servidor, así que no hay un momento de 'HTML vacío esperando datos'. Las piezas que sí son asíncronas tienen un contenedor placeholder: el navbar y el footer públicos se insertan por `fetch` dentro de un div que ya existe, y la búsqueda de jugadores muestra un texto de estado mientras espera. Esa búsqueda tiene un retardo de 350 ms desde la última tecla para no disparar una petición por letra. Y si el backend falla, el manejador global devuelve una página 500 con mensaje, no una pantalla blanca."

Sobre el preloader. La home pública tiene un precargador visual que se oculta a los 1,8 segundos. Es cosmético: no espera datos reales. No lo presentaría como una estrategia de carga.

Evidencia. `includesPublicos.js:31` y `:77` (fetch con placeholder), `buscar-jugadores-vivo.js:74-82` (debounce de 350 ms), `preloader.js`, `ManejadorErrores::responderError500`.

Límite. Si el `fetch` falla, el navbar público no aparece y solo queda un `console.error`. Debería haber un fallback estático.

IV.4. ¿Qué diferencias hay entre manipular el DOM directamente y usar un DOM virtual?

Decí esto. "El DOM real es el árbol que el navegador ya tiene; si lo tocás, el navegador recalcula layout y pintura. El DOM virtual es una copia en memoria que una librería compara con la anterior y aplica solo los cambios mínimos. Nosotros manipulamos el DOM directamente: `querySelector`, `classList`, `innerHTML`. Es suficiente para nuestros casos —mostrar resultados de búsqueda, abrir un menú, alternar un tema— y evita una dependencia y un build. La desventaja es que la consistencia queda a cargo nuestro: si se olvida un escape o un estado, el DOM queda desincronizado."

Cuándo usaría DOM virtual. En listas grandes y muy dinámicas, como un fixture que se reordena en vivo o un tablero con actualizaciones frecuentes. Ahí la reconciliación ahorra errores y repintados.

Evidencia. `publico/js/buscar-jugadores-vivo.js` (construcción de filas con `innerHTML` y escape previo), `navbar.js`, `sidebar-riel.js`, `torneo-tabs.js`.

Límite. Usamos `innerHTML` en varios lugares. En la búsqueda se escapa cada dato con una función que crea un div y lee `textContent`; en el resto son plantillas estáticas. No hay una revisión automatizada de todos los `innerHTML`.

IV.5. ¿Cómo previenes que un usuario navegue a rutas protegidas sin iniciar sesión?

Decí esto. "La protección está en el servidor, no en esconder el botón. Cada acción sensible empieza con una guarda: `requerirLogin` para sesión, `validarAdmin`, `validarOrganizador`, `validarJugador` para rol. Si no corresponde, redirige al login o devuelve acceso denegado. Además, el front controller solo despacha acciones de la lista blanca y el docroot no expone modelos ni vistas. Escribir la URL a mano no alcanza: el controlador vuelve a verificar. Hay 37 llamadas a guardas en los controladores."

Ejemplo verificable. Un organizador autenticado que intenta administrar el torneo ajeno `1200` recibe 302, no 200; el dueño sí entra. Está en el dry run.

Evidencia. `codigo_fuente/ayudantes/Sesion.php:137-205` (guardas), 37 llamadas en controladores, `index.php:38-67`, informe de verificación sección 4.3.

Límite. La guarda es manual por acción. Si se agrega una acción nueva y se olvida la línea, queda expuesta; un middleware por grupo de rutas sería más seguro.

IV.6. ¿Cómo gestionas los formularios y la validación de datos en tiempo real en el cliente?

Decí esto. "Cada formulario incluye un token CSRF oculto y se envía por POST. En el cliente usamos validación nativa de HTML5: `required`, `type=email`, longitudes mínimas; hay 76 atributos `required` en las vistas. Eso da feedback inmediato del navegador, pero no es la validación real. El servidor vuelve a validar todo, y si algo falla, re-renderiza el formulario con el mensaje y conserva lo que el usuario había escrito —hay un mecanismo de `formulario_previo` en sesión—. No hay validación en tiempo real campo por campo con JavaScript, salvo mostrar u ocultar la contraseña."

Por qué. La validación del cliente es UX; la del servidor es seguridad. Duplicar reglas en JS y PHP sin una fuente común genera divergencias.

Evidencia. 65 campos `csrf_token`, 76 `required`, `AuthControlador::procesarRegistro` (validación y política de contraseña), `PanelOrganizadorControlador::crear` (`formulario_previo`), `login.js` (solo alternar visibilidad).

Límite. No hay medidor de fortaleza ni validación en vivo de la política de contraseñas; el usuario se entera al enviar. Es una mejora pendiente, no una funcionalidad existente.

IV.7. ¿Utilizaste alguna librería de componentes o diseño propio? ¿Por qué?

Decí esto. "Diseño propio. La base es un archivo de variables CSS con colores, radios, espaciados y transiciones, y después hojas por área: pública, jugador, organizador y admin. Los íconos son Font Awesome y las tipografías Google Fonts, ambos por CDN. No usamos Bootstrap ni Tailwind: la interfaz tiene una identidad visual fuerte y una librería genérica habría peleado con ella; además, menos dependencias significa menos superficie para la CSP y menos peso."

El costo. Cada componente nuevo se escribe a mano y la consistencia depende de disciplina. Un design system formal habría ayudado cuando el equipo creció.

Evidencia. `codigo_fuente/publico/css/variables.css` (tokens), 47 hojas CSS, `docker` CSP que permite solo `cdnjs` y `fonts.googleapis.com`.

Límite. No hay catálogo de componentes ni documentación visual. Y la CSP permite estilos inline, así que parte del diseño vive en atributos `style` dentro de las vistas.

IV.8. ¿Qué son los componentes reutilizables? Da dos ejemplos de tu proyecto.

Decí esto. "Un componente reutilizable es una pieza de interfaz que se define una vez y se usa en varios lugares, con parámetros o contexto. En nuestro caso son del lado del servidor, con `include` de PHP. Dos ejemplos reales: primero, el navbar por rol —`navbar-jugador.php`, `navbar-organizador.php` y `navbar-publico.php`—, que se incluye en todas las vistas del área y mantiene la navegación coherente. Segundo, el layout del panel administrativo: `dashboard.php` recibe una variable `$vistaInyectada`, arma el `head`, el menú lateral y el contenedor, y adentro inserta cada sección. Así las secciones del admin comparten estructura sin repetir el HTML."

Tercer ejemplo, si quieren más. El `footer.php` compartido y el archivo de variables CSS, que funcionan como tokens de diseño reutilizables.

Evidencia. `codigo_fuente/vistas/componentes/navbar-*.php`, `footer.php`, `vistas/admin/dashboard.php:85-94`, `vistas/admin/plantilla/menu_lateral.php` (usado en 3 puntos).

Límite. Hay archivos placeholder vacíos —`components/alerts.js`, `components/modal.js`, `modules/bracket.js`, `modules/liga.js`, `modules/suizo.js`, `componentes/torneo-card.html`—. No los presentaría como componentes: no tienen código. La lógica de fixture y posiciones está en PHP, en `Torneo.php`.

V. Seguridad web y autenticación

V.1. ¿Qué mecanismo de autenticación usaste y cómo funciona paso a paso?

Decí esto. "Autenticación por sesión con email y contraseña. El paso a paso es: el usuario pide el login, el servidor genera un token CSRF y lo incrusta en el formulario; el POST vuelve con email, contraseña y token. El controlador valida el token, consulta si la IP está bloqueada por intentos fallidos, busca el usuario por email, verifica la contraseña con `password_verify` y comprueba que la cuenta esté activa. Si todo coincide, regenera el ID de sesión, guarda usuario, rol y vencimiento, registra el login en auditoría y redirige al panel del rol. Si falla, da un mensaje genérico y registra el intento."

Por qué regenerar el ID. Evita fijación de sesión: un atacante que conocía el ID anterior no puede reutilizarlo después del login.

Evidencia. `codigo_fuente/controladores/AuthControlador.php:47-96` (flujo), `Sesion.php:40-64` (regeneración y datos), `SeguridadAuth` (bloqueo por IP, 5 fallos en 5 minutos), `Auditoria::registrar('LOGIN_EXITOSO')`.

Límite. No hay segundo factor, ni login social, ni exigencia de email verificado: el campo `email_verificado` existe y hay endpoint de verificación, pero el login no lo exige y no se envía correo real.

V.2. ¿Cómo almacenas las contraseñas? ¿Qué algoritmo de hashing y salting usaste?

Decí esto. "Nunca en texto plano. Se guarda un hash bcrypt generado con `password_hash($contrasena, PASSWORD_BCRYPT)`. bcrypt incorpora una sal aleatoria distinta por contraseña y un costo computacional configurable; la verificación se hace con `password_verify`, que compara sin desencriptar. La columna se llama `contrasena_hash` y es `TEXT`. No hay forma de recuperar la contraseña original, solo de reemplazarla."

Detalle honesto. No fijamos un costo explícito, así que usa el valor por defecto de PHP —10—. Es razonable para un MVP; para producción conviene calibrarlo y evaluar Argon2id.

Evidencia. `codigo_fuente/modelos/Usuario.php:290` (`password_hash`), `AuthControlador.php:80` (`password_verify`), `001_ascend.sql` (`contrasena_hash`).

Límite. No hay pepper, ni política de rotación, ni historial de contraseñas aplicado en runtime: la tabla `historial_contrasenas` existe, pero el registro no la consulta.

V.3. ¿Qué es un ataque XSS y cómo lo previene tu aplicación?

Decí esto. "XSS es inyectar código —normalmente JavaScript— en una página que otros usuarios van a ver. Puede ser almacenado, si el dato queda en la base, o reflejado, si viaja en la URL. Nuestra defensa principal es escapar la salida: cada dato que viene de la base o del usuario se imprime con `htmlspecialchars` y `ENT_QUOTES`, así un `<script>` se muestra como texto y no se ejecuta. Hay 371 escapes en las vistas y no encontramos ningún `echo` directo de `$_GET` o `$_POST`. En la búsqueda AJAX, el JS escapa antes de armar el HTML. Como refuerzo, la cookie de sesión es `HttpOnly` —JavaScript no la lee— y hay una CSP que restringe orígenes de scripts."

Por qué escapar y no "limpiar". Filtrar etiquetas rompe datos legítimos; escapar en el contexto correcto es la defensa recomendada.

Evidencia. 371 `htmlspecialchars` en vistas, `buscar-jugadores-vivo.js:18-22` (función `escaparHtml`), `Sesion.php:26-35` (`httponly`), `CabecerasSeguridad.php:35-42` (CSP).

Límite. La CSP incluye `'unsafe-inline'` para scripts y estilos, lo que debilita la defensa; no hay un helper único de escape —se repite en cada vista— y no se hizo un análisis de taint completo. Es mitigación razonable, no garantía absoluta.

V.4. ¿Qué es la inyección SQL y cómo la evitaste?

Decí esto. "Es lograr que un dato del usuario se interprete como parte de la consulta SQL, por ejemplo cerrando una comilla y agregando `OR 1=1`. La evitamos con consultas preparadas de PDO en todas las operaciones con datos: hay 238 llamadas a `prepare()` en los modelos y los valores van enlazados, nunca concatenados. Además desactivamos la emulación de prepares (`ATTR_EMULATE_PREPARES = false`), así el parseo ocurre en MySQL y no en PHP. Los identificadores que no se pueden parametrizar —el orden de una lista— se eligen con un `switch` de valores permitidos; los límites se castean a entero; y los nombres de tabla que se interpolan salen de un array fijo, no de la request."

Ejemplo. La búsqueda de jugadores arma `LIKE :busquedaNombre` y pasa `'%texto%'` como parámetro. El comodín viaja como dato, no como SQL.

Evidencia. `Conexion.php:30-32` (excepciones y emulación off), 238 `prepare()`, `Equipo.php:340-360` (LIKE enlazado), `Torneo.php:2010-2030` (tablas desde array fijo), `ResultadoGlobal.php:73` (`LIMIT` casteado).

Límite. Quedan dos `query()` estáticos sin datos de usuario (`modalidades` y `sistemas_puntuacion`). No son un riesgo, pero conviene unificarlos por consistencia.

V.5. ¿Cómo gestionas los roles y permisos de usuario?

Decí esto. "Hay tres roles: 1 administrador, 2 organizador y 3 jugador. El rol se guarda en la sesión y cada acción sensible lo verifica en el servidor con una guarda: `validarAdmin`, `validarOrganizador` o `validarJugador`. Además del rol, hay control de propiedad: un organizador solo administra torneos cuyo `organizador_id` coincide con su usuario; si intenta otro, recibe una redirección. El administrador tiene un modo 'ver como' para suplantar a un usuario, que es de solo lectura: bloquea todos los POST salvo restaurar la sesión."

Matiz importante. La base tiene tablas `permisos` y `rol_permisos` con 8 filas cada una, pero el código no las consulta en runtime: la autorización es por `rol_id` hardcodeado en las guardas. Es un RBAC simple y funcional, no un motor de permisos dinámico.

Evidencia. `Sesion.php:152-205` (guardas), 37 llamadas, `index.php:73-82` (bloqueo de escritura en suplantación), `informe` sección 4.5 (POST bloqueado y 0 filas insertadas), tablas `permisos`/`rol_permisos`.

Límite. Las guardas se escriben a mano por acción; la suplantación no se audita al iniciar ni al terminar y no restringe a qué rol se puede suplantar.

V.6. ¿Cómo gestionas la expiración y renovación de sesiones o tokens?

Decí esto. "No usamos JWT ni refresh tokens: usamos sesión de PHP con vencimiento deslizante de 30 minutos. La cookie tiene `lifetime` de 1800 segundos, `HttpOnly`, `SameSite=Lax` y `Secure` solo si la request es HTTPS. En el login se regenera el ID. En cada request autenticada se compara `time()` con `expiracion_sesion`; si venció, se destruye la sesión y se manda al login; si no, se renueva la ventana por otros 30 minutos. Al cerrar sesión se destruye la sesión y se borra la cookie. El token CSRF es uno por sesión, generado con `random_bytes(32)` y comparado con `hash_equals`."

Por qué sesión y no token. El sistema es server-rendered y de un solo origen; una sesión con cookie `HttpOnly` reduce la exposición a XSS comparado con guardar un token en `localStorage`.

Evidencia. `Sesion.php:24-35` (cookie), `:40-64` (login y expiración), `:104-135` (timeboxing), `:206-230` (logout), `:233-258` (CSRF).

Límite. La sesión vive en archivos del contenedor, lo que complica el escalado horizontal; y como la demo corre por HTTP, la cookie no lleva `Secure`. El logout es GET sin token CSRF.

VI. Control de versiones, despliegue y DevOps

VI.1. ¿Cómo organizaron el flujo de trabajo en Git?

Decí esto. "Trabajamos con una rama de integración y ramas por funcionalidad: `feat/backend-mvc`, `feat/administrador-modulo`, `feat/publico-torneos`, entre otras. Cada rama se mergeaba a la de integración y los commits usan prefijos —`feat`, `fix`, `docs`— para que el historial se lea. Hubo al menos un pull request revisado y fusionado. El `.gitignore` deja fuera credenciales locales, `vendor`, los archivos subidos por usuarios, los logs y la documentación personal."

Qué no hay. No hay CI, ni protección de rama, ni revisión obligatoria. Los conflictos se resolvieron a mano y varios merges quedaron como commits de merge. Para un equipo de cuatro en un proyecto académico funcionó; para producción haría falta una política más estricta.

Evidencia. `git log --oneline` (ramas y merges visibles), `git branch -a`, `.gitignore`.

Límite. El flujo fue informal en algunos tramos: hay commits que mezclan varios cambios. No lo presentaría como Git Flow completo.

VI.2. ¿Cómo gestionan las variables de entorno en desarrollo y producción?

Decí esto. "La configuración de base de datos se lee con `getenv` y tiene valores por defecto: host, puerto, nombre, usuario, contraseña y charset. En Docker, el `docker-compose.yml` define `DB_HOST=db`, `DB_PORT=3306`, `DB_NAME=ascend`, usuario y contraseña, y `APP_URL` opcional. En XAMPP, si no hay variables, toma `localhost`, `ascend` y `root` sin contraseña. La URL base se detecta sola desde `HTTP_HOST` y `SCRIPT_NAME`, o se fuerza con `APP_URL`."

La diferencia entre entornos. La misma imagen corre en los dos lados; lo que cambia son las variables. Eso evita tener dos copias del código.

Evidencia. `codigo_fuente/configuracion/base_de_datos.php:4-9`, `configuracion/constantes.php:17-31`, `docker/docker-compose.yml:9-17`.

Límite. Las credenciales de demo están escritas en el compose, en texto plano, y no hay `.env` ni gestor de secretos. Es aceptable para una entrega académica con credenciales públicas, no para producción: ahí irían en un secret store, rotadas y con TLS.

VI.3. ¿Cuál es el procedimiento para corregir un error crítico sin interrumpir el servicio?

Decí esto. "El procedimiento que aplicamos es: reproducir el error y dejarlo registrado; crear una rama `fix/...` desde la rama de integración; corregir con el cambio mínimo y agregar o ajustar la prueba; correr la suite y el smoke test de rutas; revisar el diff; fusionar; reconstruir la imagen o reiniciar el contenedor —el código está montado como volumen, así que un `restart` toma el cambio—; verificar healthcheck y las rutas críticas. Si el cambio toca datos, antes se hace un respaldo con `mysqldump` y se aplica la migración en orden. Si falla, se revierte el commit y se reconstruye."

Qué significa "sin interrumpir". En este stack, el reinicio del contenedor web dura segundos y hay dos réplicas potenciales, pero no hay balanceador ni despliegue azul-verde. La continuidad total no está garantizada.

Evidencia. `README.md` (arranque y comandos), migraciones versionadas `001`–`007`, healthcheck en `docker-compose.yml`, suite PHPUnit y dry run documentado.

Límite. No hay CI/CD ni rollback automático. El procedimiento es manual y depende de que alguien ejecute los pasos en orden. Para producción faltaría pipeline, entorno de staging y despliegue progresivo.

VII. Metodología, gestión del proyecto y autocrítica

VII.1. ¿Cómo distribuyeron los roles y responsabilidades dentro del equipo?

Decí esto. "Nos dividimos por especialidad, no por pantalla: backend y arquitectura MVC, frontend y estilos, base de datos y modelo relacional, y seguridad, pruebas y despliegue. Cada área tenía un responsable que revisaba los cambios de las otras, y las decisiones que afectaban a todos —contrato de datos, rutas, convenciones de nombres— se acordaban antes de implementar. Usamos Discord para coordinación y GitHub para el código. La documentación de planificación registra esa asignación y también preveía rotación de roles."

Por qué por especialidad. El proyecto cruza cinco materias; dividir por capas permitió que cada uno pudiera explicar y defender su parte, y evitó que todos tocaran todo sin dueño.

Evidencia. `Documentos_Juan/DocumentacionProgramacion.md` sección 1.4 (acuerdos de kickoff: asignación de roles Backend, Frontend, DBA, QA) y sección 1.1 (equipo y restricciones).

Límite. La división no fue rígida: hubo solapamiento y cambios de último momento. No hay registros de horas por persona que permitan medir la carga real.

VII.2. ¿Qué requerimientos funcionales y no funcionales definieron y cuáles cumplieron al 100%?

Decí esto. "Definimos un baseline de 74 requisitos funcionales, agrupados en nueve módulos, y 29 no funcionales. Los funcionales van desde autenticación y equipos hasta torneos en liga, eliminación directa y

suizo, resultados, consulta pública y administración. Los no funcionales cubren responsive, integridad, validación, manejo de errores, hashing, autorización, portabilidad con Docker, MVC, respaldos y privacidad."

Qué está cubierto y qué no. "Cumplidos y verificados: autenticación y roles, equipos y solicitudes, los tres formatos de torneo, carga de resultados y posiciones, catálogo público y rankings, panel de administración, controles de seguridad, Docker y migraciones. No están al 100%: recuperación de contraseña, envío real de email y verificación obligatoria, bye en el sistema suizo, auditoría de todas las acciones y permisos dinámicos. El objetivo SMART decía 100%; la entrega cubre el alcance implementable y declara esas exclusiones en el README."

Por qué no inflo el número. Porque el jurado puede pedir la recuperación de contraseña y el sistema muestra "no disponible en esta versión". Declararlo antes es más sólido que negarlo.

Evidencia. `DocumentacionProgramacion.md:38` y `:242-255` (74 RF / 29 RNF y módulos), matriz RF→tabla→modelo→controlador→vista en la misma documentación, `README.md` sección "Estado de entrega y limitaciones".

Límite. No hay una matriz de cumplimiento con porcentaje por requisito. La cobertura se puede defender por módulo y por ruta verificada, no con un "100%" global.

VII.3. Si tuvieran que empezar de cero, ¿qué cambiarían?

Decí esto. "Tres cosas, en orden de impacto. Primero, congelar antes el contrato de datos y la semilla: al principio el esquema y los datos de prueba cambiaron varias veces y eso obligó a rehacer pantallas y migraciones. Segundo, tener CI desde el día uno: correr pruebas y el dry run en cada push, en lugar de descubrir regresiones al final. Tercero, centralizar validación y escape: hoy cada controlador valida a su manera y cada vista escapa a su manera; un helper único habría evitado inconsistencias. Y una cuarta, específica del proyecto: preparar la VM Linux antes para poder ejecutar el toolkit de sistemas operativos, que quedó validado solo en sintaxis."

Qué no cambiaría. "El MVC propio y el renderizado del servidor. Nos obligó a entender el flujo y hoy podemos explicar cada capa. Un framework habría sido más rápido, pero menos defendible en esta materia."

Evidencia. Migración `003` no idempotente y correcciones posteriores `005`/`006`; `ayudantes/Validador.php` vacío; informe de aceptación del toolkit con ejecución pendiente.

Límite. Es una autocrítica basada en lo que pasó, no una proyección medida. No hay datos de cuánto tiempo se habría ahorrado.

Anexo A. Preguntas de seguimiento probables

"¿Qué es un front controller y qué ventaja tiene?" Un único punto de entrada que centraliza seguridad, errores y despacho. Ventaja: una sola frontera que auditar. Costo: la lista de rutas se mantiene a mano.

"El Singleton es criticado, ¿por qué lo usaron?" Porque garantiza una conexión por request sin pasarla por parámetro. La crítica es el estado global y el acoplamiento para testear; en este proyecto se acepta por simplicidad y se prueba con base desechable.

"¿Qué pasa si dos administradores editan el mismo torneo?" Gana el último que guarda; no hay bloqueo optimista con versión. En operaciones críticas —fixture, cambios de estado, resultados— sí hay transacción y `FOR UPDATE`.

"¿Por qué no hay una API REST?" Porque el cliente es el mismo servidor que renderiza. El endpoint AJAX devuelve JSON, pero no hay contrato ni versionado. Separarla sería el paso siguiente si apareciera una app móvil.

"¿Qué pasa si el usuario desactiva JavaScript?" Las páginas siguen funcionando porque vienen renderizadas, pero el navbar y el footer públicos no se inyectan. Es una deuda concreta.

"¿Cómo probarían 1.000 usuarios simultáneos?" Con una herramienta de carga contra el stack, midiendo latencia y errores, y observando MySQL. No lo hicimos; el análisis actual es arquitectónico, no medido.

"¿Qué es un índice compuesto y dónde lo usarían?" Un índice sobre varias columnas en orden. En `torneo_encuentros` serviría `(torneo_id, estado, fecha)` para listar pendientes de un torneo sin escanear la tabla.

"¿Qué es CSRF y cómo lo evitan?" Un sitio externo fuerza al navegador autenticado a enviar un POST. Lo evitamos con token de sesión en cada formulario, comparado con `hash_equals`, y rechazo 419. Dos acciones GET que cambian estado quedan como deuda.

"¿La contraseña se puede recuperar?" No. Solo se reemplaza por otra; el hash es irreversible. La recuperación automática no está implementada.

"¿Qué falta para producción?" HTTPS, secretos fuera del compose, `display_errors` apagado, CSP sin `unsafe-inline`, permisos dinámicos aplicados, auditoría completa, CI/CD, respaldo y restauración probados, y pruebas de carga.

Anexo B. Límites declarados

#	Límite	Consecuencia	Estado
1	Sin paginación por offset	Listados grandes no escalan	Deuda declarada
2	Sin API REST ni OpenAPI	No hay cliente externo formal	Fuera de alcance
3	Validación centralizada inexistente (`Validador.php` vacío)	Reglas repetidas y posibles divergencias	Deuda técnica
4	Sin recuperación de contraseña ni email real	El usuario no puede autogestionar el acceso	Fuera de alcance
5	Login no exige email verificado	Cualquiera con email válido entra	Brecha conocida
6	`permisos`/`rol_permisos` sin uso en runtime	RBAC por rol fijo, no dinámico	Deuda de diseño

#	Límite	Consecuencia	Estado
7	Suplantación sin auditoría ni restricción de rol destino	No queda traza de “ver como”	Brecha conocida
8	Dos GET que cambian estado sin CSRF (`torneoAdmin/eliminar`, `juego/cambiarEstado`)	Riesgo CSRF y borrado accidental	Deuda de seguridad
9	CSP con `unsafe-inline`; sin HTTPS; credenciales demo públicas	Mitigación parcial de XSS y transporte sin cifrar	Entorno académico
10	Sin CI/CD, sin E2E; 7 tests / 25 assertions	Cobertura automatizada baja	Deuda declarada
11	Sin APM ni slow query log; sesión en archivos	Diagnóstico manual y escalado con fricción	Deuda operativa
12	Archivos JS/HTML placeholder vacíos	No aportan funcionalidad	Limpieza pendiente
13	Sin `php.ini` endurecido	`display_errors` no fijado en producción	Deuda de despliegue

Anexo C. Comandos para mostrar evidencia en vivo

```
# Estado del stack
docker compose -f docker/docker-compose.yml ps

# Respuesta y tiempo de una ruta
curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" \
  "http://localhost:8080/index.php?c=torneo&a=catalogoPublico"

# Conteos de la base
docker compose -f docker/docker-compose.yml exec -T db mysql -uroot -proot123 -N -
e \
  "SELECT COUNT(*) FROM information_schema.tables WHERE table_schema='ascend';
   SELECT COUNT(*) FROM information_schema.referential_constraints WHERE
constraint_schema='ascend';"

# Políticas de borrado
docker compose -f docker/docker-compose.yml exec -T db mysql -uroot -proot123 -N -
e \
  "SELECT DELETE_RULE, COUNT(*) FROM information_schema.referential_constraints
   WHERE constraint_schema='ascend' GROUP BY DELETE_RULE;"

# Consulta y plan
docker compose -f docker/docker-compose.yml exec -T db mysql -uroot -proot123 -e \
  "EXPLAIN SELECT * FROM ascend.torneos WHERE estado='inscripciones_abiertas';"

# Evidencia estática
grep -ro "htmlspecialchars" codigo_fuente/vistas | wc -l
grep -ro "prepare(" codigo_fuente/modelos | wc -l
```

```
grep -rn "FOR UPDATE" codigo_fuente | wc -l
grep -rn "OFFSET" codigo_fuente | wc -l

# Historia de Git
git log --oneline -15
```

Anexo D. Frases que conviene usar y evitar

No decir	Decir
"Tiene paginación."	"No hay paginación; los listados están acotados por dominio y con `LIMIT`."
"Es 100% seguro."	"Tiene controles para un MVP: CSRF, escape, bcrypt, guardas y cabeceras; no es producción."
"Usamos RBAC completo."	"Autorización por rol con guardas; las tablas de permisos existen pero no se consultan."
"Escapamos todo."	"371 escapes en vistas y ningún `echo` directo de `$_GET`/`$_POST`; falta un helper único."
"La sesión escala."	"La app escala; la sesión en archivos es el primer obstáculo."
"Probamos todo."	"50 rutas con 200, 7 tests y un dry run; la concurrencia y la carga no están medidas."
"Tiene recuperación de contraseña."	"Está declarada como no disponible en esta versión."
"Eso no lo vimos."	"No lo medimos; está declarado como límite."

Anexo E. Fuentes internas

- `README.md` — arranque, configuración, estado de entrega y limitaciones.
- `informes_auditoria/VERIFICACION_FINAL_DRY_RUN.md` — rutas, pruebas, aislamiento y límites.
- `Documentos_Juan/DocumentacionProgramacion.md` — alcance, requisitos, riesgos, decisiones y matriz RF.
- `Documentos_Juan/RequisitosFuncionales.md` y `RequerimientosNoFuncionales.md` — baseline de 74 RF y 29 RNF.
- `base_de_datos/migraciones/001_ascend.sql` — 3FN, herencia y políticas de integridad.
- `codigo_fuente/publico/index.php` — front controller, whitelist y bloqueo de suplantación.
- `codigo_fuente/core/ManejadorErrores.php` y `CabecerasSeguridad.php` — errores y cabeceras.
- `codigo_fuente/ayudantes/Sesion.php` — sesión, expiración y CSRF.
- `codigo_fuente/modelos/Conexion.php` — PDO, Singleton y prepares reales.
- `codigo_fuente/modelos/Torneo.php` y `Equipo.php` — transacciones, bloqueos y reglas.
- `docker/docker-compose.yml`, `docker/Dockerfile`, `docker/apache-vhost.conf` — despliegue.